Moore's Law

$L_t = 0.7 L_{t-2}$

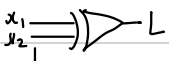
$A_t = L_t^2$

XO

$L = \bar{x}_1 x_2 + x_1 \bar{x}_2$

$= x_1 \oplus x_2$

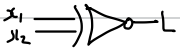
V: $x_1 \wedge x_2$



XNOR

$L = \bar{x}_1 \bar{x}_2 + x_1 x_2$

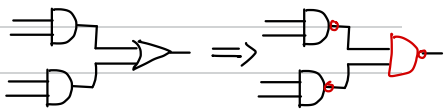
$= \overline{x_1 \oplus x_2}$



n-input XOR

$\begin{cases} L = 1, & \text{odd \# of high input} \\ L = 0, & \text{even} \ldots \end{cases}$

SOP → NAND-NAND



Cannonical SOP

1. Identify rows with 1
2. Write out minterms
   $\bar{x}$ if 0, x if 1
   ex. $001 \rightarrow \bar{x}_2 \bar{x}_1 x_0$
3. OR them up
   ex. $x_2 \bar{x}_1 + x_2 x_1$

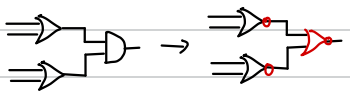
Cannonical POS

1. Identify rows with 0
2. Write out maxterms
   $\bar{x}$ if 1, x if 0
   ex. $001 \rightarrow x_2 + x_1 + \bar{x}_0$
3. AND them up
   ex. $(x_2 + \bar{x}_1)(\bar{x}_2 + \bar{x}_1)$

Common Identities

Base form	Dual form
$x + x \cdot y = x$	$x \cdot (x + y) = x$
$x + \bar{x} y = x + y$	$x(\bar{x} + y) = xy$
$\overline{x_1 \cdot x_2 \cdot x_3 \cdots} = \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \cdots$	$\overline{x_1 + x_2 + x_3 \cdots} = \bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3 \cdots$
$(x \cdot y) + (\bar{x} \cdot z) + (y \cdot z)$	$(x + y) \cdot (\bar{x} + z) \cdot (y + z)$
$= (x \cdot y) + (\bar{x} \cdot z)$	$= (x + y) \cdot (\bar{x} + z)$

POS → NOR-NOR



Reduction

&a

= a[0] & a[1] & ⋯

Verilog Syntax

Concatenation

y = {a[2:1], b[2:0]}

= a[2] a[1] b[2] b[1] b[0]

K-maps

Step 1: Find PIs (circle 2^n cells) — covers a minterm no other PI does

Step 2: Identify essentials PIs and include in the cover

Step 3: Choose the suitable PIs, so that
1. all minterms are covered
2. cost is minimized (minimizing literals)

- Notes:
  - ↳ Circle "overlappingly" to find all PIs →
  - ↳ We can circle 0s (maxterms) to create a POS instead.

if blocks

```
always_comb
begin
  if(⋯)
  else(⋯)
end
```

case statement

```
always_comb
begin
  case(logic)
    2'b00: x = ⋯
    ⋮
    default: x = 1'b0⋯
  end case
end
```

No modules instantiation inside case/always comb

Parameters

```
module adder
#(parameter WIDTH = 8)
input logic[WIDTH-1:0] a;
```

Shifting

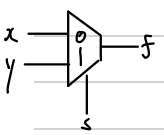
LS: a<<b

RS: a>>b

shift a by b bits

Multiplexer (mux)

Block



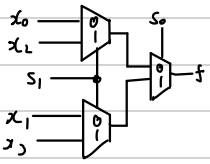
Logic

$f = \bar{s} x + s y$

Verilog

```
module mux2to1(input logic x,y,s
               output logic f);
assign f = (~s & x)|(s & y);
endmodule
```

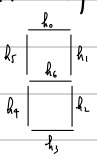
4 to 1 mux



$f = \bar{s}_1 \bar{s}_0 x_0 + \bar{s}_1 s_0 x_1 + s_1 \bar{s}_0 x_2 + s_1 s_0 x_3$

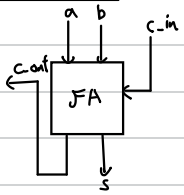
$= s_0 (\bar{s}_1 x_1 + s_1 x_3) + \bar{s}_0 (\bar{s}_1 x_0 + s_1 x_2)$

HEX disp.



- 0 when lit up
- 1 when off

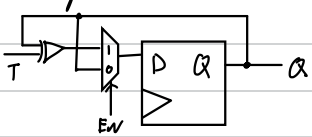
Full adder



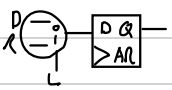
$s = a \oplus b \oplus c_in$

$c_out = b\,c_in + a\,c_in + ab$

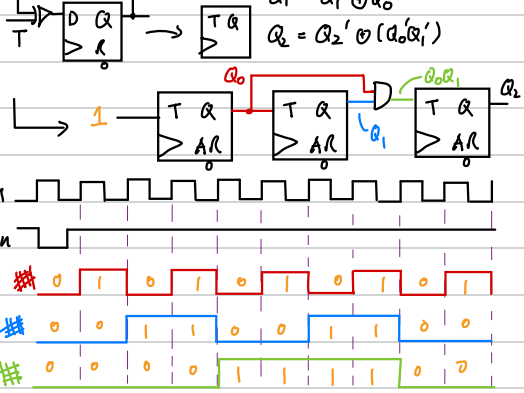
Pausing (EN)



Parallel Load



TQ FF and Counters

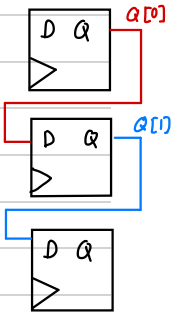


$Q_0 = 1 \oplus Q_0'$

$Q_1 = Q_1' \oplus Q_0'$

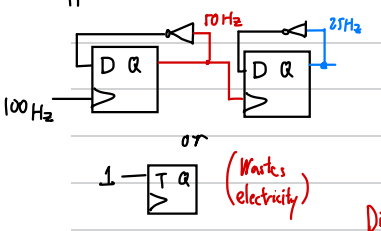
$Q_2 = Q_2' \odot (Q_0' Q_1')$

Shift Register



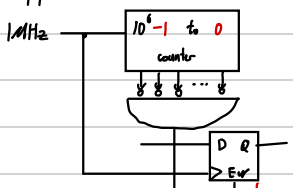
Rate dividers

Approach 1: FFs



(Wastes electricity)

Approach 2: Counters



Divide rate by n → count from $\frac{f_{clk}}{n} - 1$

Verilog (Parallel load, enable, async reset)

```
module upcount(input logic [3:0] R, //load value
               input logic resetn, clk, EN, L
               output logic [3:0] Q);
always_ff @(posedge clk, negedge resetn) begin
  if(!resetn) Q <= 4'b0; //Reset highest priority
  else if(L) Q <= R; //Load value
  else if(EN) Q <= Q + 1; end
end module
```

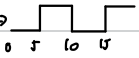
.do file

force <object_name> <value>, set input <object_name> to value, ex force {SW[0]} 0

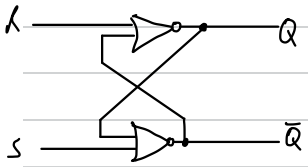
force <object_name> <value 1> <time 1> <value 2> <time 2> - r <time 3>

ex force {clk} 0 0ns, 1 {5ns} -r 10ns
1. 2. 3.

1. set clk to 0 after 10ns
2. set clk to 1 after 5ns
3. Repeat after 10 ns



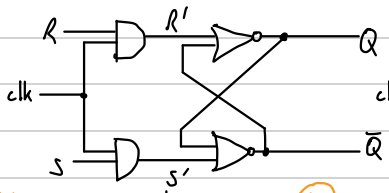
① R (Reset) S (Set) Latch



S	R	Q	$\bar{Q}$	
0	0	%	%	Store
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Not used

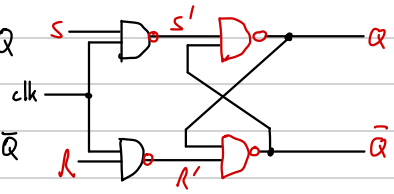
↳ Undetermined if $(S,R) = (1,1) \rightarrow (0,0)$

② Gated RS Latch

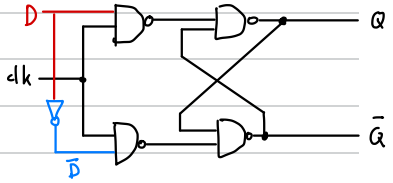


S	R	clk	Q(t+1)
X	X	0	Q(t)
0	0	1	Q(t)
0	1	1	0
1	0	1	1
1	1	1	not used

③ NAND-Gated RS



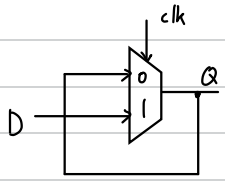
④ Gated D-Latch



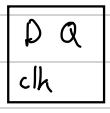
D	clk	Q(t+1)	
X	0	Q(t)	Store
0	1	0	Reset
1	1	1	Set

↳ Eliminates $S = R$

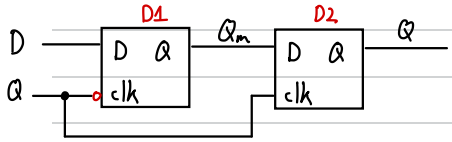
⑤ Mux Gated D-Latch



Latch Block



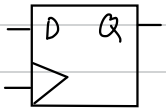
Flip Flops (FF)



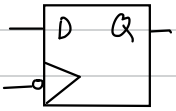
$clk = 0 \rightarrow D_1: Q_m = D,\ D_2: Q = prev$

$clk \uparrow \rightarrow Q = Q_m = D$

Pos-edge FF



Neg-edge FF



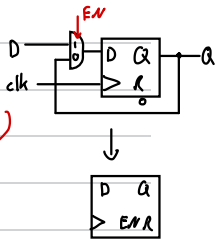
D-latch Verilog

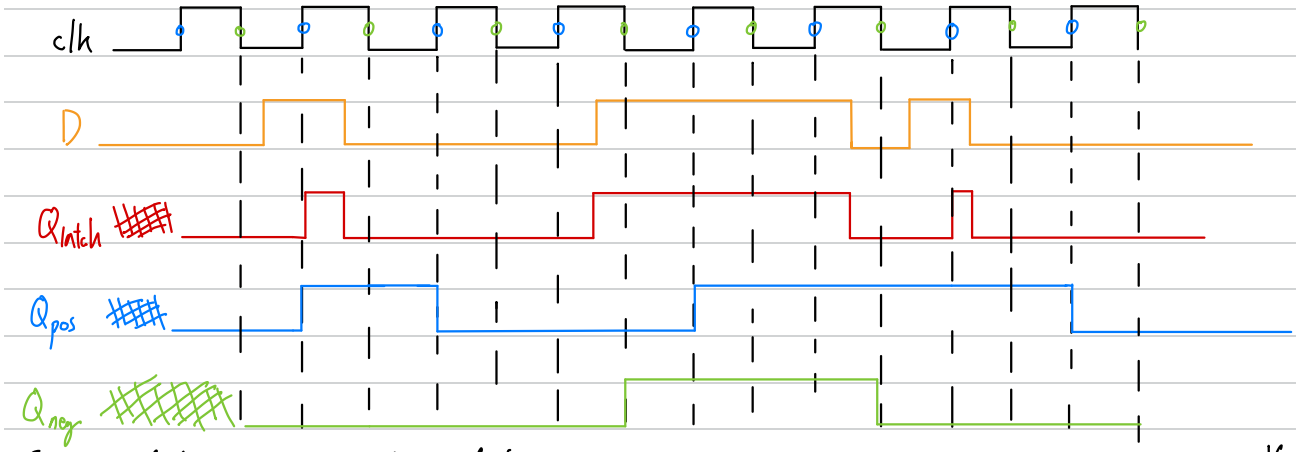
```
module D_latch (input logic D, clk,
                output logic Q);
always_latch
if (clk==1)
Q=D; //stores Q otherwise
endmodule
```

Verilog for FFs

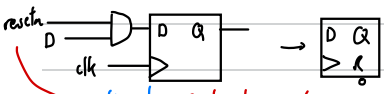
```
module D_FF (input logic D, clk
             output logic Q);
always_ff @(posedge clk)
Q <= D;
```

Enable





Synchronous Resets

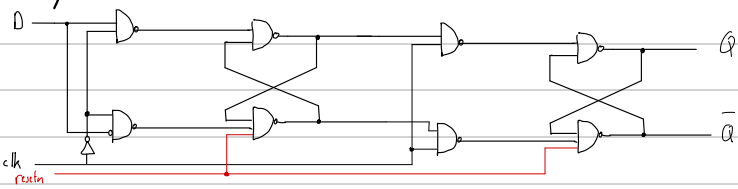


active low, resets when resetn = 1

Verilog

```
if (!resetn) Q <= 1'b0;
```

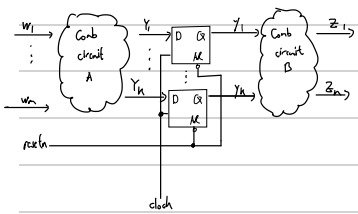
A-sync Resets



Verilog

```
always_ff @(posedge clk,
            negedge resetn)
begin
if(!resetn) Q <= 1'b0;
else Q<=D;
```

FSM



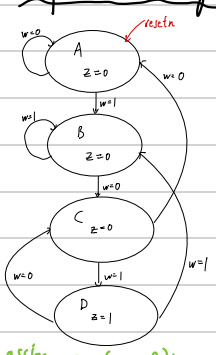
Verilog for FSM

```
typedef enum logic [1:0] (A,B,C,D) statetype;
statetype ps,ns;
always_ff(...)
ps <= ns;
always_comb
case(ps)
A: if(w) ns=B;
   else ns=A; ...
assign z = (ps==D);
```

FSM Steps

(ex. from lecture 9, z = 1 if a sequence of w = 1,0,1 occurs, z = 0 otherwise)

Step 1: state diagram



Step 2: State Table

PS	NS w=0	NS w=1	z
A	A	B	0
B	C	B	0
C	A	D	0
D	C	D	1

Step 3: State assignment

- Choose # of flip flops (chose 2 this case $y_2 y_1$)
- Choose state codes (A: 00, B: 01, C: 10, D: 11)

Step 4: State-assigned table (Step 2 again)

PS $y_2 y_1$	NS w=0 $Y_2 Y_1$	NS w=1 $Y_2 Y_1$	z
0 0	0 0	0 1	0
0 1	0 1	0 1	0
1 0	0 0	1 1	0
1 1	1 0	0 1	1

Step 5: Synthesize circuit

↳ Express NS, output (z) in logic

↳ Draw circuit w/ flip flops